

PROJECT EVALUATION REPORT

CSE476 CA1: Habit Streak Tracker Personal Habit Agent
An Implementation and Loop Design Analysis

Course & Assignment:	CSE476 CA1 Project 1
Project Title:	Habit Streak Tracker: Personal Habit Agent
Author:	Himanshu Kumar
GitHub Repository:	https://github.com/HIMANSHUgg75/habit-tracker
Live Demonstration:	https://tokyo-lowest-granted-logs.trycloudflare.com
Development Status:	Completed / Demonstration and Deployment Successful
Report Generated:	September 4, 2026

Executive Summary

This evaluation report reviews the development and architecture of the Habit Streak Tracker, a lightweight personal habit agent that records daily activity, calculates current and longest streaks, and compares consistency across habits. The project demonstrates a transparent tool-driven workflow, persistent JSON-backed state, a Python implementation, an interactive React interface, and a FastAPI service. The system treats a calendar date as the unit of truth, preventing duplicate logs on the same day from inflating progress.

The project includes two core tools, **log_habit(name)** and **get_streak(name)**, plus the **most_consistent()** summary operation. A browser interface provides an accessible way to log habits and inspect progress, while the command-line agent and notebook provide demonstration and evaluation paths.

1. Core Architecture & Tech Stack

The Habit Streak Tracker is structured as a small Python application with a FastAPI HTTP layer and a React frontend. The implementation separates decision logic, habit persistence, tool operations, and presentation so each part can be tested independently.

Python: Provides the local tools, date arithmetic, persistence, and offline agent demonstration.

Tool-driven agent loop: agent.py selects and dispatches operations, returns observations, and continues until the task is complete.

FastAPI: Exposes habit logging, streak retrieval, summary, and health-oriented API routes.

React and CSS: Provides the live dashboard for adding habits, viewing streak cards, and identifying the most consistent habit.

Jupyter notebook: Provides an interactive demonstration and evaluation record for the agent workflow.

Project Codebase Structure

File / Module	Functionality & Role in Agent System
agent.py	Runs the plan-act interaction loop and routes requested operations to local tools.
tools.py	Defines log_habit, get_streak, and most_consistent; handles date normalization and streak calculations.
habits.json	Stores habit names and ISO calendar dates so progress survives restarts.
backend/server.py	FastAPI service exposing the habit API and serving the production React build.
frontend/src/App.js	React dashboard for logging habits and viewing current and best streaks.
demo.ipynb	Interactive notebook demonstration of the agent and tool behavior.

2. The Plan-Act Execution Loop

The agent does not rely only on an opaque function-calling engine. It exposes a clear plan-act sequence: interpret the habit request, select a tool, execute the local operation, capture the observation, and produce a concise result. This makes the workflow easy to inspect and test.

Each operation is deterministic at the tool boundary. The model or command-line driver can choose an action, but the Python tools remain responsible for date handling, duplicate detection, streak arithmetic, and persisted state.

The web deployment uses the same backend routes as the local application. The production React build is served from FastAPI, allowing the UI and API to share one public origin and avoiding a browser dependency on localhost.

User Input

Natural-language habit goal, such as logging a reading session or checking the current streak.

Plan Step

The agent identifies the requested habit operation and its required name.

Act Step

The dispatcher invokes `log_habit`, `get_streak`, or `most_consistent`.

Observation

The tool returns structured streak data and updates `habits.json` when needed.

Final Synthesis

The agent reports the result, including duplicate-log status when relevant.

Detailed Workflow Walkthrough

1. User Input: The loop begins with a request to record or inspect a habit.
2. Plan Step: The agent determines which tool matches the user's intent and prepares its arguments.
3. Act Step: The selected Python tool reads or updates the JSON-backed habit history.
4. Observation Step: The tool returns the current streak, longest streak, status, or summary.
5. Duplicate Protection: A second log for the same calendar date is reported as already counted.
6. Final Synthesis: The agent presents a human-readable result and ends the loop.

3. Tool Design & Memory Management Details

The agentic behavior of the Habit Streak Tracker depends on small local tools. The agent routes user intent to these operations, while the persistence layer bridges separate runs and keeps the history available after restart.

Registered Capabilities (Tools)

Tool 1: `log_habit(name)`

Records today's ISO date for the named habit, calculates the current streak, and returns whether today's entry was already present.

Tool 2: `get_streak(name)`

Reads the saved history and returns the current streak, all-time best streak, and a status description.

Tool 3: `most_consistent()`

Compares all stored habits and identifies the strongest current performer.

Persistence: `habits.json`

Stores data as a mapping from habit names to ISO date lists. Dates are normalized and deduplicated before calculations.

Development Challenge & Anchoring Solution

Honest Failure Encountered: The first implementation treated repeated logs on the same day as separate progress, which could inflate the apparent streak.

Resolution: The implementation now treats the calendar date as the unit of truth. It deduplicates stored dates and returns an `already_logged_today` signal, preserving accurate streaks while keeping the user informed.

4. Evaluation, Failure Analysis & Future Directions

An evaluation of the Habit Streak Tracker reveals strengths in transparent tool execution, simple persistence, deterministic date arithmetic, and a focused user interface, while also identifying clear future improvements.

System Strengths

Explicit Execution Visibility: The agent loop exposes the selected operation and returned observation, making behavior easy to debug.

Deterministic Streak Math: Date normalization, duplicate protection, and consecutive-day calculations are handled by Python tools rather than generated text.

Persistent Local State: habits.json keeps user history available across process restarts without requiring a database for the core demonstration.

Accessible Deployment: The React interface and FastAPI routes are served together, providing one working URL for the demonstration.

System Limitations & Failure Risk Analysis

Single-file storage: JSON persistence is appropriate for a local demonstration but is not designed for concurrent multi-user production traffic.

Temporary public URL: The current live link is a quick tunnel and depends on the local computer and running service remaining online.

Recommended Enhancements (Future Scope)

1. Replace the JSON file with SQLite or a hosted database for concurrent and multi-user persistence.
2. Deploy the frontend and backend to permanent hosting with environment-specific configuration.
3. Add authentication, per-user habit collections, reminders, and calendar history visualization.

Code Repository Reference

The complete codebase, tests, notebook demonstrations, production frontend build, and FastAPI service are hosted publicly on GitHub.

Repository Link: <https://github.com/HIMANSHUgg75/habit-tracker>

Demonstration Link: <https://tokyo-lowest-granted-logs.trycloudflare.com>